Generic vs. non-generic Poseidon2 and Fiat–Shamir

Benchmark report, `prover-ray/crypto/genericbench`

28 October 2026

1 Summary

- The fully generic Poseidon2 code is about 7–10× slower than the non-generic baseline on KoalaBear and about 3× slower on MamaBear. KoalaBear's gap is larger because its baseline uses gnark-crypto's AVX-512 assembly.
- The code the prover runs today (`current`: the generic hasher and transcript over gnark-crypto's compression) is on par with the baseline

Run	Code	Sizes	Samples
1 (Tables 1, 2)	`3ce6cc6e5`, fully generic	`full`, minus FS rounds=4096/update=16384	6×500 ms
2 (Table 3, "initial")	`6b3d84ad3`, branches and the three paths	`medium`, KoalaBear only	6×200 ms
3 (Tables 3, 4)	run 2 + gnark-crypto diagonal replicated	`medium`, both fields	6×200 ms
4 (Table 5)	same code as run 3	CompressParallel, KoalaBear, 1–192 threads	6×200 ms

The single skipped FiatShamir case took about 40 s per operation in the generic code and dominated the runtime. Tables report the median of 6 samples. Unless noted otherwise, every difference against the baseline is statistically significant according to `benchstat` ($p = 0.002$). The exceptions are `current` on MDHasher $n = 2^{20}$ in run 1 ($p = 0.065$), on FiatShamir rounds=16/update=8 in run 3 ($p = 0.065$) and on MDHasher $n = 4096$ in run 3 ($p = 0.026$). All are differences of under 2%. Run 2 survives only as the medians in `report/data/paths-run2-medians.csv`; its raw output was lost.

3 Background: what is benchmarked

All the benchmarks are built on the **Poseidon2 permutation**, the core of the Poseidon2 hash. It takes a fixed-size state of 16 or 24 field elements and scrambles it into another state of the same size. It is reversible, hence "permutation". It runs 27 rounds (6 full, 21 partial), each made of three steps:

1. **Add round keys**: add fixed constants derived from a seed.
2. **S-box**: cube each element, $x \mapsto x^3$. This is the nonlinear step that gives the hash its security. Full rounds cube every element; partial rounds cube only the first.
3. **Linear mixing**: multiply the state by a fixed matrix, so every element affects every other. Full rounds use the "external" matrix built from 4×4 blocks (M4); partial rounds use the cheaper "internal" matrix $\mathbf{1}\mathbf{1}^T + D$.

The other benchmark families are built on the permutation:

- **Compress**: two 8-element blocks form a width-16 state, which is permuted; the second block is then added back (feed-forward). **CompressParallel** runs the same with one goroutine per hardware thread (192 threads on 96 cores).
- **MDHasher**: hashes n elements, 8 at a time, with one compression per chunk.
- **FiatShamir**: a protocol-shaped transcript. Each round absorbs `upd` elements, samples a digest, samples query indices and derives a named challenge.

The permutation dominates every benchmark, so its speed carries through to all of them.

4 Results 1: fully generic code

Table 1: KoalaBear, fully generic code (before any branching). Ratios are time relative to `baseline`.

Benchmark	baseline	fully generic	ratio	`current`	ratio	allocs `baseline`	allocs fully generic	allocs `current`
Permutation (width=16)	570 ns	5.18 µs	9.09×	–	–	0	222	–
Permutation (width=24)	706 ns	7.43 µs	10.5×	–	–	0	298	–
Compress	585 ns	5.22 µs	8.91×	–	–	0	224	–
CompressParallel	4.31 ns	967 ns	224×	–	–	0	224	–
MDHasher (n=8)	733 ns	4.96 µs	6.77×	627 ns	0.86×	5	225	1
MDHasher (n=64)	4.99 µs	39.3 µs	7.87×	4.88 µs	0.98×	5	1 793	1
MDHasher (n=4096)	308 µs	2.52 ms	8.20×	311 µs	1.01×	5	114 689	1
MDHasher (n=65536)	4.91 ms	40.1 ms	8.16×	4.96 ms	1.01×	5	1 835 009	1
MDHasher (n=1048576)	78.2 ms	626 ms	8.01×	78.8 ms	1.01×	5	29 360 129	1
FiatShamir (rounds=16, upd=8)	388 µs	2.79 ms	7.19×	394 µs	1.02×	254	126 171	731
FiatShamir (rounds=16, upd=1024)	1.61 ms	12.8 ms	7.93×	1.63 ms	1.01×	254	581 342	731
FiatShamir (rounds=16, upd=16384)	20 ms	164 ms	8.17×	20.2 ms	1.01×	255	7 462 669	732
FiatShamir (rounds=256, upd=8)	6.18 ms	44.6 ms	7.22×	6.31 ms	1.02×	3 849	2 018 586	11 526
FiatShamir (rounds=256, upd=1024)	25.8 ms	204 ms	7.92×	26 ms	1.01×	3 850	9 301 340	11 528
FiatShamir (rounds=256, upd=16384)	320 ms	2.61 s	8.14×	323 ms	1.01×	3 874	119 402 596	11 550
FiatShamir (rounds=4096, upd=8)	99.5 ms	708 ms	7.12×	101 ms	1.02×	65 518	32 301 300	188 400
FiatShamir (rounds=4096, upd=1024)	412 ms	3.2 s	7.77×	417 ms	1.01×	65 536	148 825 042	188 420

How to read the tables. Each ratio is time relative to `baseline`: 1× is the same speed, 9× means nine times as long, and below 1× is faster. The allocs columns give heap allocations per operation.

Findings.

Table 2: MamaBear, fully generic code. The MamaBear baseline is gnark-crypto's (non-assembly) permutation under the generic hasher.

Benchmark	baseline	fully generic	ratio	allocs `baseline`	allocs fully generic
Permutation (width=16)	2.09 µs	7.27 µs	3.48×	0	250
Permutation (width=24)	3.39 µs	10.1 µs	2.98×	0	340
Compress	2.2 µs	7.26 µs	3.30×	2	252
CompressParallel	211 ns	1.42 µs	6.72×	2	252
MDHasher (n=8)	2.12 µs	6.71 µs	3.16×	3	253
MDHasher (n=64)	17.2 µs	53.7 µs	3.13×	17	2 017
MDHasher (n=4096)	1.13 ms	3.45 ms	3.06×	1 025	129 025
MDHasher (n=65536)	18 ms	54.3 ms	3.01×	16 385	2 064 385
MDHasher (n=1048576)	290 ms	820 ms	2.83×	262 145	33 030 145
FiatShamir (rounds=16, upd=8)	1.29 ms	3.82 ms	2.95×	1 788	141 789
FiatShamir (rounds=16, upd=1024)	5.78 ms	17.5 ms	3.03×	5 852	653 858
FiatShamir (rounds=16, upd=16384)	73.6 ms	223 ms	3.04×	67 300	8 395 370
FiatShamir (rounds=256, upd=8)	20.6 ms	61.2 ms	2.97×	28 439	2 268 465
FiatShamir (rounds=256, upd=1024)	92.5 ms	279 ms	3.02×	93 472	10 461 576
FiatShamir (rounds=256, upd=16384)	1.17 s	3.56 s	3.03×	1 076 630	134 325 810
FiatShamir (rounds=4096, upd=8)	330 ms	945 ms	2.86×	458 573	36 298 855
FiatShamir (rounds=4096, upd=1024)	1.47 s	4.31 s	2.93×	1 499 070	167 388 320

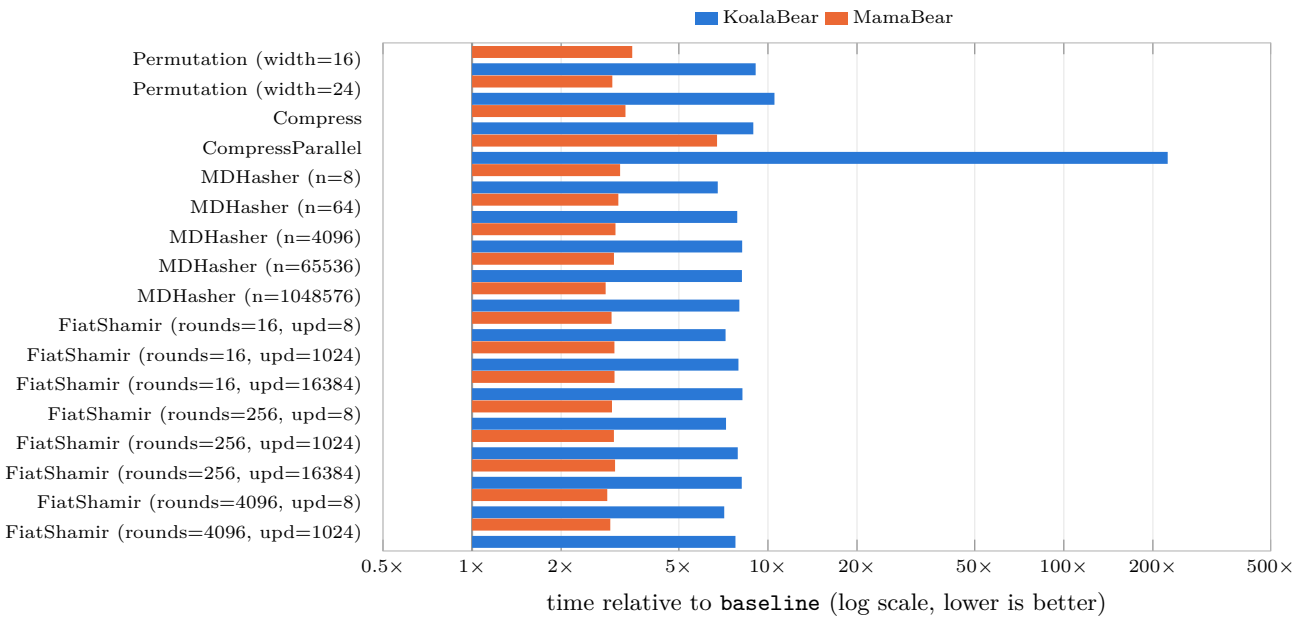


Figure 1: Fully generic code: time relative to each field's `baseline` (Tables 1 and 2). The vertical line marks parity.

- KoalaBear `generic` is 6.8–10.5× slower than baseline, and MamaBear `generic` 2.8–3.5×. Without assembly on the MamaBear side, the MamaBear figure is the better measure of the cost of genericity itself.
- `current` is on par with baseline: from 14% faster (MDHasher, $n = 8$) to about 2% slower (FiatShamir). It also makes 80% fewer allocations in MDHasher, but about 2.9× more in FiatShamir. The absolute numbers are small (731 vs. 254 at 16 rounds), but this may deserve a look.
- The generic code allocates heavily: 222–340 allocations per permutation, about 28 per absorbed element in the KoalaBear MDHasher, and up to 167 million per FiatShamir operation.

Why the generic code is slow. Go does not compile a generic function separately for every type. It compiles it once per "GC shape" (here, every `[1]uint32`-shaped field shares the code) and passes a dictionary of the type's methods. So inside `sBox`, `matMulM4` and the other helpers, each `PT(x).Mul`, `PT(x).Add`, ... is an *indirect call* through that dictionary. The compiler cannot inline it, and it cannot see what the callee does with the pointers it receives, so it moves the operands to the heap. That causes both the per-operation call overhead and the hundreds of allocations per permutation.

Example: the internal matrix. Every version below implements the same step of a partial round, multiplying the state by $\mathbf{1}\mathbf{1}^T + D$: add up all elements, then set each element to the sum plus itself times its diagonal entry. This is the fully generic version. Each `PT(...)` call goes through the dictionary, and `sum` and `tmp` are moved to the heap:

Listing 1: Fully generic `matMulInternal` (commit `3ce6cc6e5`).

```
func (h *Permutation[T, PT]) matMulInternal(input []T) {
	var sum T
	for i := range input {
		PT(&sum).Add(&sum, &input[i]) // indirect call
	}
	var tmp T
	for i := range input {
		PT(&tmp).Mul(&input[i], &h.params.InternalDiag[i]) // indirect call
		PT(&input[i]).Add(&sum, &tmp) // indirect call
	}
}
```

For comparison, gnark-crypto's own scalar fallback (used when there is no AVX-512) is written for one field and one width. It never multiplies by the diagonal: the diagonal entries are small values such as ± 2, $\pm\frac{1}{2}$ or 2^{-8}, so it uses doublings, halvings and shifts instead. Since run 3, the generic code and the KoalaBear branch use the same formula (Section 5.1); Tables 1 and 2 predate that change.

Listing 2: gnark-crypto KoalaBear `matMulInternalInPlace`, width 16 (excerpt).

```
var sum fr.Element
sum.Set(&input[0])
for i := 1; i < h.params.Width; i++ {
	sum.Add(&sum, &input[i])
}
// mul by diag16: [-2, 1, 2, 1/2, 3, 4, -1/2, -3, -4, 1/2^8, ...]
var temp fr.Element
input[0].Sub(&sum, temp.Double(&input[0]))
input[1].Add(&sum, &input[1])
input[2].Add(&sum, temp.Double(&input[2]))
temp.Set(&input[3]).Halve()
input[3].Add(&sum, &temp)
// ...
input[9].Add(&sum, temp.Mul2ExpNegN(&input[9], 8))
// ...
```

5 Results 2: branching to concrete KoalaBear code in the hot loops

Each hot helper of the permutation (`sBox`, `addRoundKey`, `matMulExternal` with M4, `matMulInternal`) now starts with `if h.kb { ...; return }`. The flag `kb` is computed once in `NewPermutation` by a type check. The KoalaBear branch reinterprets its operands as `[]koalabear.Element` (via `unsafe`, without copying) and runs concrete loops that the compiler can inline and keep on the stack. The check runs once per helper call; putting it inside the per-element loops would only add work. Other fields keep the generic code.

Listing 3: `matMulInternal` with the hot-loop branch (`crypto/poseidon2`). The vector case belongs to Section 6.

```
// permutation.go: the generic helper branches once, on entry.
func (h *Permutation[T, PT]) matMulInternal(input []T) {
	if h.kb { // set once in NewPermutation: is T koalabear.Element?
		if h.kbPath == KoalaBearVector {
			kbMatMulInternalVec(asKB(input), asKB(h.params.InternalDiag))
			return
		}
		kbMatMulInternal(asKB(input))
		return
	}
	// ... generic code for every other field ...
}

// koalabear.go: reinterpret []T as []koalabear.Element, without copying.
func asKB[T any](s []T) []koalabear.Element {
	return unsafe.Slice((*koalabear.Element)(unsafe.Pointer(unsafe.SliceData(s))), len(s))
}

// koalabear.go: gnark-crypto's code, copied; direct calls the compiler inlines.
func kbMatMulInternal(input []koalabear.Element) {
	var sum koalabear.Element // stays on the stack
	sum.Set(&input[0])
	for i := 1; i < len(input); i++ {
		sum.Add(&sum, &input[i])
	}
	var temp koalabear.Element
	input[0].Sub(&sum, temp.Double(&input[0]))
	input[1].Add(&sum, &input[1])
	// ... as in Listing 2, for width 16 and 24 ...
}
```

Table 3: KoalaBear, the hot-loop branches before (run 2) and after (run 3) replicating gnark-crypto's internal-matrix diagonal. Each ratio is relative to the `baseline` of the same run; the baseline column shows run 3.

Benchmark	baseline	branches (initial)	ratio	branches, gnark diagonal	ratio	allocs, gnark diagonal
Permutation (width=16)	570 ns	1.92 µs	3.37×	1.46 µs	2.57×	0
Permutation (width=24)	706 ns	2.63 µs	3.73×	2.08 µs	2.94×	0
Compress	585 ns	2.03 µs	3.46×	1.54 µs	2.62×	2
CompressParallel	4.42 ns	170 ns	38.6×	147 ns	33.2×	2
MDHasher (n=64)	4.97 µs	16.2 µs	3.26×	11.8 µs	2.38×	17
MDHasher (n=4096)	308 µs	1.03 ms	3.34×	792 µs	2.57×	1 025
MDHasher (n=65536)	4.91 ms	16.6 ms	3.37×	12.6 ms	2.57×	16 385
FiatShamir (rounds=16, upd=8)	134 µs	381 µs	2.86×	296 µs	2.21×	698
FiatShamir (rounds=16, upd=1024)	1.36 ms	4.5 ms	3.33×	3.45 ms	2.54×	4 762
FiatShamir (rounds=256, upd=8)	2.11 ms	6.1 ms	2.89×	4.8 ms	2.27×	11 013
FiatShamir (rounds=256, upd=1024)	21.7 ms	71.9 ms	3.31×	55.4 ms	2.56×	76 046

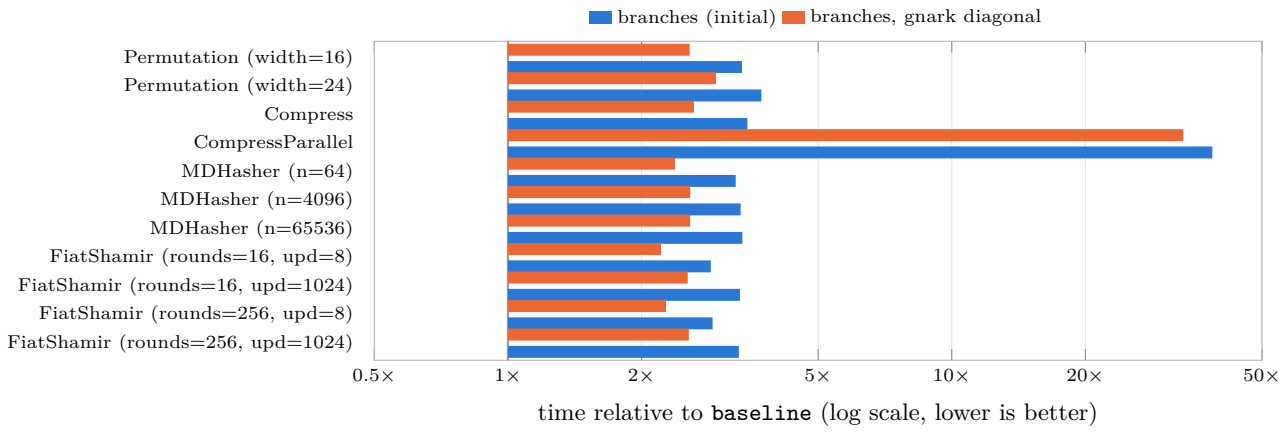


Figure 2: KoalaBear hot-loop branches, time relative to `baseline`, before and after replicating gnark-crypto's internal-matrix diagonal (Table 3).

Findings. The permutation is now allocation-free. Its first version (run 2) was 3.4–3.7× slower than baseline. With gnark-crypto's internal-matrix formula replicated (run 3, Section 5.1), it is 2.6–2.9× slower, and the hashing and Fiat–Shamir benchmarks 2.2–2.6×. The remaining gap is the AVX-512 assembly: one AVX-512 instruction processes 16 elements at once, which is exactly one width-16 state, and gnark-crypto's assembly keeps the whole state in registers for all 27 rounds. Scalar Go code cannot do that. The remaining allocations (2 per compression) are analysed in Section 7.

5.1 Replicating gnark-crypto's optimisations

A comparison with gnark-crypto should measure the cost of genericity, not of different formulas. Each helper was therefore checked against gnark-crypto's scalar code for both fields:

Step	gnark-crypto's scalar code	Our code
S-box, round keys, M4, external matrix	same formulas	identical
Internal matrix: diagonal	doublings, `Halve`, `Mul2ExpNegN` instead of `Mul`	replicated in run 3 (generic and KoalaBear branch)
Internal matrix: sum, MamaBear only	raw 64-bit sum, a single reduction at the end (`sumState`)	**not replicated**

To replicate the diagonal in the generic code, `generic.Element` gained two methods that both fields already have, `Halve` and `Mul2ExpNegN`. The KoalaBear branch is a line-by-line copy of gnark-crypto's function. The generic version applies the 2^{-n} entries in a short loop over a table of shifts rather than unrolling them.

The effect differs by path. The KoalaBear branch became about 24% faster (Table 3). The fully generic MamaBear permutation did not: 7.69 µs in run 3, compared with 7.27 µs in run 1, a difference within the noise of those runs. In the generic code every field operation is an indirect call, and the formula replaces one multiplication by two to four cheaper operations, so the number of calls, which dominates, goes up. MamaBear's `sumState` relies on the field's internal representation and cannot be expressed through `generic.Element`; replicating it needs a MamaBear branch or a new interface method.

6 Results 3: vector operations vs. assembly

Two ways to close the assembly gap were implemented side by side. The choice is made with `SetKoalaBearPath`:

- `KoalaBearScalar` (default, column "generic + hot-loop branches"): the concrete loops of Section 5.
- `KoalaBearVector` (`generic-vec`): the same branches, but the round keys, the full S-box layer and the internal matrix use gnark-crypto's batched `koalabear.Vector` operations (`Add`, `Mul`, `Sum`), which run AVX-512 loops over slices. M4 and the external matrix stay scalar because they mix neighbouring elements. For width 24, the last 8 elements fall back to scalar code because the vector operations work in blocks of 16.
- `KoalaBearAsm` (`generic-asm`): the branch sits at the top of `Permute` and hands the whole permutation to gnark-crypto's AVX-512 function.

Listing 4: The vector path: `matMulInternal` as three batched calls (`koalabear.go`).

```
func kbMatMulInternalVec(input, diag []koalabear.Element) {
	v := koalabear.Vector(input)
	sum := v.Sum() // AVX-512 call 1
	var buf [maxWidth]koalabear.Element // stack scratch
	sums := koalabear.Vector(buf[:len(input)])
	for i := range sums {
		sums[i] = sum
	}
	v.Mul(v, diag) // AVX-512 call 2
	v.Add(v, sums) // AVX-512 call 3
}
```

Listing 5: The assembly path: the branch sits at the top of `Permute`, so no helper runs at all (`permutation.go`).

```
func (h *Permutation[T, PT]) Permute(input []T) error {
	// ...
	if h.kbAsm != nil { // set by SetKoalaBearPath(KoalaBearAsm)
		return h.kbAsm.Permutation(asKB(input)) // gnark-crypto: all 27 rounds in AVX-512
	}
	h.matMulExternal(input)
	// ... rounds, each calling the helpers of Listing 3 ...
}
```

Table 4: KoalaBear, all code paths side by side (run 3, `medium` scale). The hot-loop branches include the gnark-crypto internal-matrix diagonal.

Benchmark	baseline	generic + hot-loop branches	ratio	`generic-vec`	ratio	`generic-asm`	ratio	`current`	ratio	allocs `generic-asm`
Permutation (width=16)	570 ns	1.46 µs	2.57×	11.6 µs	20.3×	574 ns	1.01×	–	–	0
Permutation (width=24)	706 ns	2.08 µs	2.94×	12.3 µs	17.5×	704 ns	1.00×	–	–	0
Compress	585 ns	1.54 µs	2.62×	11.7 µs	19.9×	655 ns	1.12×	–	–	2
CompressParallel	4.42 ns	147 ns	33.2×	475 ns	107×	77.4 ns	17.5×	–	–	2
MDHasher (n=64)	4.97 µs	11.8 µs	2.38×	93.1 µs	18.7×	5.26 µs	1.06×	4.86 µs	0.98×	17
MDHasher (n=4096)	308 µs	792 µs	2.57×	5.96 ms	19.4×	335 µs	1.09×	310 µs	1.01×	1 025
MDHasher (n=65536)	4.91 ms	12.6 ms	2.57×	95.3 ms	19.4×	5.34 ms	1.09×	4.97 ms	1.01×	16 385
FiatShamir (rounds=16, upd=8)	134 µs	296 µs	2.21×	2.08 ms	15.6×	141 µs	1.06×	133 µs	0.99×	698
FiatShamir (rounds=16, upd=1024)	1.36 ms	3.45 ms	2.54×	25.9 ms	19.1×	1.49 ms	1.10×	1.37 ms	1.01×	4 762
FiatShamir (rounds=256, upd=8)	2.11 ms	4.8 ms	2.27×	33.3 ms	15.8×	2.27 ms	1.08×	2.14 ms	1.01×	11 013
FiatShamir (rounds=256, upd=1024)	21.7 ms	55.4 ms	2.56×	413 ms	19.0×	23.7 ms	1.09×	21.9 ms	1.01×	76 046

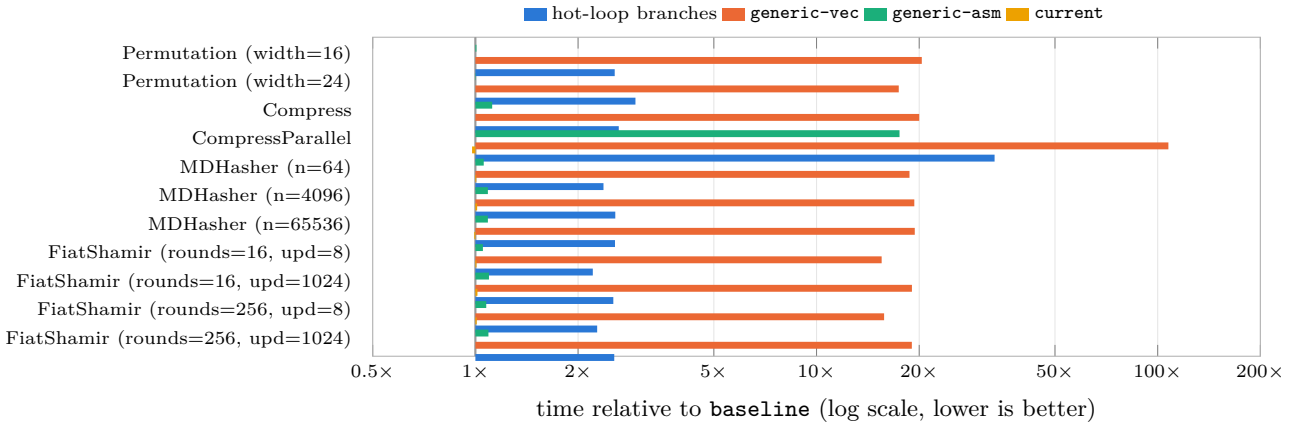


Figure 3: KoalaBear code paths, time relative to `baseline` (Table 4). `current` exists only for MDHasher and FiatShamir.

Findings.

- `generic-asm` matches the baseline permutation (1.00–1.01×). The Compress, MDHasher and FiatShamir benchmarks remain 6–12% slower than baseline. That overhead comes from the generic code around the permutation.
- `generic-vec` is about 20× slower than baseline, and about 8× slower than the scalar branch it was meant to improve. (It still multiplies by the diagonal; it was not updated in run 3.) A CPU profile shows almost all the time inside the vector routines (75% in the 21 partial rounds), at about 150 ns per call on 16 elements. The vector routines are designed for long arrays; on a 16-element state the cost of each call outweighs the work. The likely cause is the penalty the CPU pays when switching between AVX-512 and ordinary instructions, but this has not been confirmed.

Why the two cannot be combined in one permutation. Both paths live in the same codebase and are compiled into the same binary; nothing conflicts. A given permutation can use only one of them, though: the assembly function performs all 27 rounds in a single call, so once `Permute` hands it the state, none of the per-step helpers run and there is nowhere for a vector branch to apply. gnark-crypto does not expose the individual rounds of its assembly. This is the underlying trade-o□: branching as low as possible and matching the baseline's speed pull in opposite directions, because the fast code exists only as one whole-permutation block.

7 Why CompressParallel is so slow

Even with the assembly permutation, `CompressParallel` is about 17× slower than baseline at 192 threads (Table 4). To find out why, it was re-run at 1 to 192 threads, with the Go garbage collector (GC) on and o□, and profiled at 192 threads. The machine has 2 sockets of 48 cores, each core running 2 hardware threads, so 192 threads means 96 physical cores. The data is in `report/data/`.

7.1 Where the allocations come from

A memory profile attributes every allocation to two lines of the generic `Compress` (`crypto/poseidon2/hasher.go`):

Listing 6: Generic `Compress`. The two marked arrays are moved to the heap.

```
func Compress[T any, PT generic.Element[T]](perm *Permutation[T, PT], state, block Octuplet[T]) Octuplet[T]
    {
```

```
        var x [CompressionWidth]T // ALLOCATES: 16 elements, 64 bytes
        copy(x[:BlockSize], state[:])
        copy(x[BlockSize:], block[:])
        if err := perm.Permute(x[:]); err != nil { // x escapes here
                panic(err)
        }
        res := block // ALLOCATES: 8 elements, 32 bytes
        for i := range res {
                PT(&res[i]).Add(&res[i], &x[BlockSize+i]) // res escapes here
        }
        return res
}
```

Listing 7: gnark-crypto `vortex.CompressPoseidon2`: the same steps, nothing allocated.

```
func CompressPoseidon2(a, b Hash) Hash {
        res := Hash{}
        var x [16]koalabear.Element
        copy(x[:], a[:])
        copy(x[8:], b[:])
        copy(res[:], x[8:])
        if err := compressPerm.Permutation(x[:]); err != nil {
                panic(err)
        }
        for i := range res {
                res[i].Add(&res[i], &x[8+i])
        }
        return res
}
```

That is 2 allocations and 96 bytes per compression, for every generic path. These are fixed-size local arrays and should stay on the stack, as they do in gnark-crypto's almost identical `CompressPoseidon2`. They escape for the reason given in Section 4: `x` is passed to `perm.Permute` and `res` to `PT(&res[i]).Add`, both indirect calls the compiler cannot see through, so it assumes the pointers may be kept and moves both arrays to the heap.

7.2 Scaling with the number of threads

Table 5: KoalaBear CompressParallel, nanoseconds per compression (lower is better) by number of threads (`-cpu`), with the garbage collector on (default) and o (`GOGC=off`). Ratios are relative to `baseline` at the same thread count. `baseline` allocates nothing, so the GC setting does not a ect it.

		generic-asm				generic + hot-loop branches			
Threads	baseline	GC on	ratio	GC off	ratio	GC on	ratio	GC off	ratio
1	585	645	1.1×	626	1.1×	1572	2.7×	1620	2.8×
2	293	337	1.2×	315	1.1×	799	2.7×	854	2.9×
4	146	178	1.2×	159	1.1×	400	2.7×	399	2.7×
8	73.4	101	1.4×	81.5	1.1×	214	2.9×	193	2.6×
16	36.8	71.1	1.9×	40.7	1.1×	125	3.4×	99.8	2.7×
32	19.0	67.3	3.5×	21.9	1.2×	96.3	5.1×	50.5	2.7×
48	13.9	76.3	5.5×	15.2	1.1×	98.5	7.1×	36.8	2.6×
96	7.4	86.1	12×	9.5	1.3×	106	14×	23.4	3.1×
144	5.6	83.6	15×	7.9	1.4×	117	21×	21.7	3.9×
192	4.6	75.8	16×	7.1	1.5×	123	27×	18.2	3.9×

Table 5 shows the key result:

- **With the GC on**, `generic-asm` keeps up with baseline only up to about 8 threads. It reaches its best time at 32 threads (67 ns) and then gets *slower* as threads are added (76–86 ns from 48 threads on). Baseline keeps improving to 4.6 ns, so the ratio grows from 1.1× to 16×. The branched scalar path behaves the same way (best 96 ns at 32 threads, 27× at 192).
- **With the GC o**, `generic-asm` scales almost as well as baseline: 1.1–1.5× at every thread count, 7.1 ns at 192 threads. The scalar path stays at 2.6–3.9×, close to its single-threaded ratio.

So the parallel slowdown is caused almost entirely by **garbage collection**, not by allocation itself and not by the permutation. Allocating without collecting costs 10–50% (the remaining gap with the GC o).

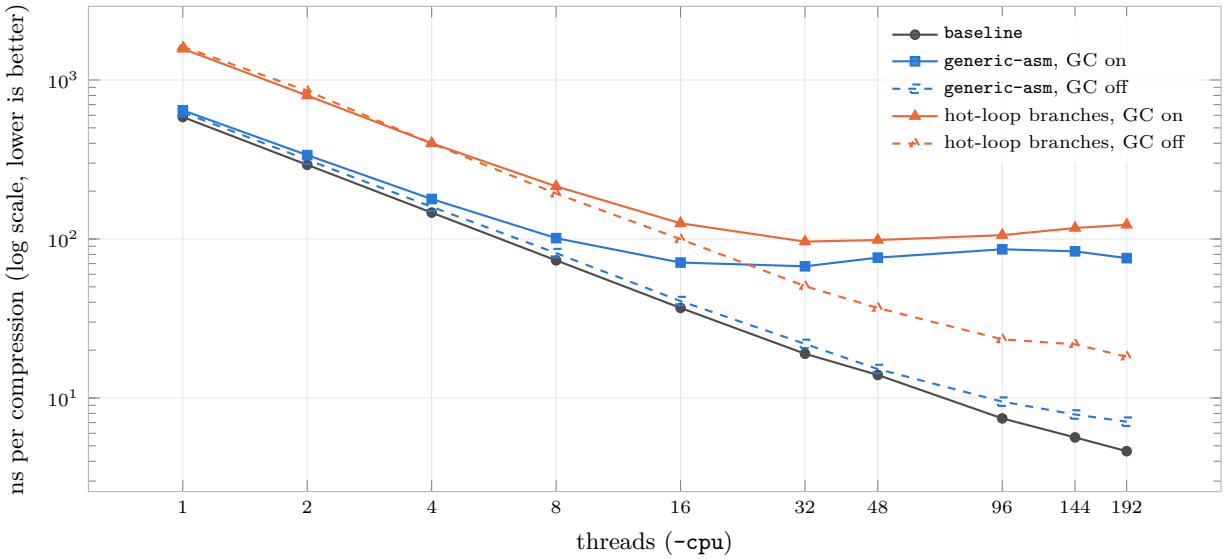


Figure 4: KoalaBear CompressParallel scaling (Table 5). Solid: garbage collector on (default); dashed: o□. Colour follows the implementation.

7.3 Why garbage collection hurts this much

A CPU profile of `generic-asm` at 192 threads (2 s run, taken before run 3; the assembly path does not use the code changed in run 3) shows:

- **289 GC cycles** during the run with the GC on, about one every 20 ms. With the GC o□ there were 13, all forced by the test framework between runs.
- **Less than half the CPU in use.** With the GC on, the profile recorded 95 CPU-seconds over the run; with it o□, 203 CPU-seconds over the same duration, and 4.1× more compressions completed. The di□erence is time threads spent stopped or waiting, which a CPU profile does not show directly.
- Of the CPU time that was used with the GC on, about 7% went to background marking (`gcBgMarkWorker`), 4% to mark assists (`gcAssistAlloc`), 11% to allocation (`mallocgc`), and the rest of the runtime share to locking, spinning and scheduling (`lfstack.pop`, `futex`, `procyield`, `schedule`).

The mechanism:

1. **The live heap is tiny** (about 4 MB). With the default `GOGC=100`, a GC cycle starts whenever the heap doubles, here every ~5 MB allocated.
2. **192 threads allocate fast**: 96 bytes per compression at about 15 million compressions per second is about 1.5 GB/s, so a new cycle starts every few milliseconds.
3. **Each cycle costs every thread.** A cycle stops the world twice, which means waiting for all 192 threads to reach a safe point. During marking, a quarter of the threads are taken for background mark workers, and threads that allocate are made to help with marking (assists). Synchronising all of this across 192 threads and two sockets dominates.
4. **More threads make it worse.** Adding threads raises the allocation rate, so cycles come more often and each one has more threads to coordinate. Past about 32 threads, extra threads add more GC cost than useful work.

7.4 Caveats

- The benchmark is close to a worst case. In the prover the live heap is much larger, so with `GOGC=100` cycles are much rarer (but each one marks more). The real impact depends on the prover's heap and its other allocations. The fix is the same either way: make `Compress` allocation-free.
- Above 96 threads the extra threads are hyperthreads sharing a core. Even baseline gains only 1.7× from 96 to 192 threads.
- The profiled runs used a 2 s benchtime and are slower than the table (profiling a 192-thread process has overhead). The comparison between GC on and o□ is between runs with identical settings.
- The same two allocations account for the 16 385 allocations of MDHasher at n = 65 536, so they also matter single-threaded at a smaller scale (6–12%).

8 Recommendations

1. Make `KoalaBearAsm` the default when the CPU supports AVX-512 and the parameters are 6 full and 21 partial rounds; otherwise fall back to the scalar branch.
2. Remove the two allocations in `Compress` by applying the same KoalaBear branch there: reinterpret `x` and `res` as KoalaBear arrays and do the feed-forward with direct calls. A per-permutation scratch buffer would not work, as it would break concurrent use.
3. Drop `KoalaBearVector`, or keep it only as a recorded negative result.
4. For MamaBear (about 3.5×, with no assembly available), the overhead is purely the generic code. Closing it means concrete branches like KoalaBear's, which would also be the place to replicate gnark-crypto's `sumState` (Section 5.1).
5. Look into the higher allocation count of `current` in FiatShamir (about 2.9× baseline).

A Reproducing

From `prover-ray/`, with `benchstat` installed (`go install golang.org/x/perf/cmd/benchstat@latest`):

```
# all fields, medium scale (a few minutes)
scripts/bench-generic-poseidon.sh medium <outdir>

# KoalaBear only, as in run 3 (about 80 s)
go test ./crypto/genericbench/ -run TestAgreement
GENERIC_BENCH_SCALE=medium go test ./crypto/genericbench/ -run '^$' \
  -bench '/field=koalabear/' -benchmem -count 6 -benchtime 200ms > raw.txt
benchstat -col /impl raw.txt
```